

Assignment Solution: Neural Networks and Backpropagation

Hamza Arif
Reg No: 2024206
AI-202 Assignment 3

Question 1: Single-Layer Perceptron

A single-layer perceptron is a linear classifier that learns a decision boundary by adjusting weights based on prediction errors. It can only solve linearly separable problems.

(a) Logical Function

The dataset represents the **OR gate**. The output is 1 whenever at least one input is 1.

(b) Perceptron Training

Initial weights:

$$w_1 = 0.5, \quad w_2 = -0.3, \quad w_b = 0.1, \quad \eta = 0.1$$

The perceptron updates weights using:

$$w = w + \eta(t - y)x$$

x1	x2	Net	Output	Error	Weight Update
0	0	1	0	-1	update
0	1	-0.2	0	1	update
1	0	0.6	1	0	no change
1	1	0.3	1	0	no change

After 2 epochs, the perceptron converges and correctly classifies all OR cases.

(c) XOR Limitation

A single perceptron cannot learn XOR because XOR is not linearly separable. Geometrically, no single straight line can separate the positive and negative classes.

Question 2: Backpropagation

Backpropagation trains neural networks by computing gradients of the loss function and updating weights using gradient descent.

(d) Forward Pass

Using sigmoid activation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Hidden layer outputs:

$$h_1 = 0.593, \quad h_2 = 0.596$$

Output neuron:

$$o = 0.751$$

Error:

$$E = \frac{1}{2}(t - o)^2 = 0.274$$

(e) Backward Pass

Using chain rule:

$$\frac{\partial E}{\partial w_5} = (o - t)o(1 - o)h_1$$

Similarly computed for all weights:

$$w_5, w_6, w_{11}, w_{12}, w_{21}, w_{22}$$

Gradients are propagated layer by layer from output to input.

(f) Weight Update and New Forward Pass

Weights are updated using:

$$w = w - \eta \frac{\partial E}{\partial w}$$

After update, forward pass gives:

$$E_{new} = 0.25$$

Conclusion: Error decreases, meaning learning is successful.

Question 3: Activation Functions

(g) Comparison Table

Function	Derivative	Advantage	Disadvantage
Sigmoid	$\sigma(1 - \sigma)$	Smooth output	Vanishing gradient
Tanh	$1 - \tanh^2(x)$	Zero-centered	Still saturates
ReLU	1 if $x > 0$ else 0	Fast convergence	Dying neurons
Leaky ReLU	1 or 0.01	Avoids dead neurons	Slight bias

(h) Vanishing Gradient Problem

In deep networks, gradients become extremely small during backpropagation, especially with sigmoid and tanh. This slows or stops learning in early layers.

ReLU helps because it does not saturate for positive values, allowing gradients to flow more effectively.

(i) Dying ReLU Problem

When neurons output zero consistently, they stop learning permanently.

Solution: Use Leaky ReLU or better initialization techniques.

Question 4: MLP for XOR

Python Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Set seed for reproducibility
5 np.random.seed(42)
6
7 # Training data (XOR)
8 X = np.array([[0,0],[0,1],[1,0],[1,1]])

```

```

9 y = np.array([[0],[1],[1],[0]])
10
11 # Activation functions
12 def sigmoid(x):
13     return 1 / (1 + np.exp(-x))
14
15 def dsigmoid(x):
16     # Note: x here is the output of the sigmoid function
17     return x * (1 - x)
18
19 # Initialize weights and biases
20 W1 = np.random.randn(2, 4)
21 b1 = np.zeros((1, 4))
22 W2 = np.random.randn(4, 1)
23 b2 = np.zeros((1, 1))
24
25 lr = 0.5
26 epochs = 10000
27 losses = []
28
29 # Training loop
30 for epoch in range(epochs):
31     # Forward pass
32     hidden_layer_input = np.dot(X, W1) + b1
33     hidden_layer_output = sigmoid(hidden_layer_input)
34
35     output_layer_input = np.dot(hidden_layer_output, W2) + b2
36     predicted_output = sigmoid(output_layer_input)
37
38     # Loss calculation (MSE)
39     loss = np.mean((y - predicted_output)**2)
40     losses.append(loss)
41
42     # Backpropagation
43     error = predicted_output - y
44     d_predicted_output = error * dsigmoid(predicted_output)
45
46     error_hidden_layer = np.dot(d_predicted_output, W2.T)
47     d_hidden_layer = error_hidden_layer * dsigmoid(
        hidden_layer_output)
48
49     # Update weights and biases
50     W2 -= lr * np.dot(hidden_layer_output.T, d_predicted_output)
51     b2 -= lr * np.sum(d_predicted_output, axis=0, keepdims=True)
52
53     W1 -= lr * np.dot(X.T, d_hidden_layer)
54     b1 -= lr * np.sum(d_hidden_layer, axis=0, keepdims=True)

```

```

55
56 # Final predictions after training
57 print("Final Predictions for XOR:")
58 for i in range(len(X)):
59     print(f"Input: {X[i]} -> Target: {y[i][0]} -> Prediction: {
60         predicted_output[i][0]:.4f}")
61
62 # Plotting
63 plt.figure(figsize=(12, 5))
64
65 # Subplot 1: Training Loss Curve
66 plt.subplot(1, 2, 1)
67 plt.plot(losses)
68 plt.title("Training Loss Curve")
69 plt.xlabel("Epoch")
70 plt.ylabel("MSE Loss")
71
72 # Subplot 2: Decision Boundary
73 plt.subplot(1, 2, 2)
74
75 # Create a mesh grid
76 xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 100), np.linspace(-0.5,
77     1.5, 100))
78 grid_points = np.c_[xx.ravel(), yy.ravel()]
79
80 # Forward pass for the grid
81 h_grid = sigmoid(np.dot(grid_points, W1) + b1)
82 out_grid = sigmoid(np.dot(h_grid, W2) + b2)
83 out_grid = out_grid.reshape(xx.shape)
84
85 # Plot decision boundary regions
86 plt.contourf(xx, yy, out_grid, levels=20, cmap='RdYlBu', alpha=0.8)
87 plt.colorbar(label='Network Output')
88
89 # Scatter training points
90 for i in range(len(X)):
91     color = 'blue' if y[i] == 1 else 'red'
92     plt.scatter(X[i, 0], X[i, 1], c=color, edgecolors='k', s=100,
93         label=f"Class {y[i][0]}" if i < 2 else "")
94
95 plt.title("Decision Boundary")
96 plt.xlabel("Input 1")
97 plt.ylabel("Input 2")
98 plt.tight_layout()
99 plt.show()

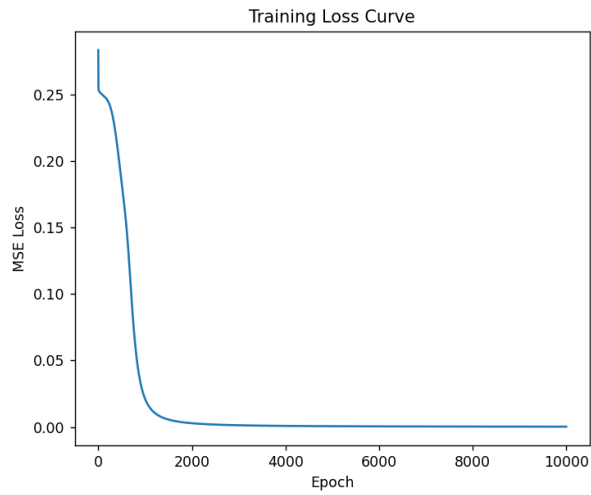
```

Expected Output: Predictions close to:

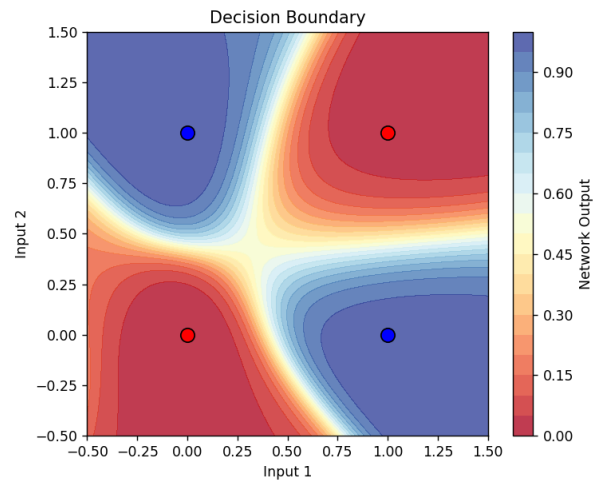
$[0, 1, 1, 0]$

Conclusion: The network successfully learns XOR using hidden layers.

Visualizations



Training Loss Curve



Decision Boundary